

Form validation rules review

Exported on 9/3/2026 at 01:07:39 GMT+5 from Cursor (3.18.9)

User

Scan all the forms in the application and see which validation rules are required and where and why? Also, see which fields should be required and which should be optional.

Cursor

I'll scan both frontend and backend for forms and validation schemas to map required vs optional fields and where they're enforced.

Verifying frontend/backend mismatches on a few critical forms.

Form Validation Audit — SatLife Application

This audit covers ~40+ forms across the frontend (satlife-frontend-main) and backend (satlife-backend-main). Validation is split across three layers:

- 1. **Frontend** — manual useState + inline if checks + toast.error() (no Zod/react-hook-form in use despite being installed)
- 2. **Backend** — FastAPI + Pydantic/SQLModel schemas (422 on type/required failures) + service-layer business rules (400 with custom messages)
- 3. **Database** — nullable=False , unique indexes, FK constraints

Validation Architecture

Layer	Mechanism	Notes
-------	-----------	-------

Shared form component	<code>components/entity-form.tsx</code>	Required-only check: <code>if (field.required && !formData[field.name])</code>
Inventory helper	<code>hooks/use-inventory-entity-form.ts</code> → <code>validateCreate()</code>	Context-aware (inventory vs hierarchy, serialized vs bulk)
Password policy	<code>lib/password-policy.ts</code>	Fetches from <code>GET /auth/password-policy</code> ; enforced on signup and user create
Entity list registry	Backend <code>entity_list_service.py</code>	Hardware/inventory names must exist in Settings → Entity List
Backend schemas	<code>app/schemas/schemas.py</code> + <code>app/models/base.py</code>	Source of truth for API contracts

Important: `react-hook-form`, `zod`, and `@hookform/resolvers` are installed but **unused** in application forms. Validation is scattered and inconsistent.

1. Authentication & Users

Login (`app/login/page.tsx`)

Field	Required?	Why
<code>username</code>	Yes	Identity for OAuth2 login
<code>password</code>	Yes	Authentication credential

Backend: same via OAuth2 form. Failures: invalid credentials, locked account, inactive user, expired password.

Signup / Request Access (`app/signup/page.tsx`)

Field	Required?	Why
<code>full_name</code>	Yes	User display identity
<code>username</code>	Yes	Unique login identifier

password	Yes	Must pass password policy (min 8 chars, upper/lower/number; special if enabled)
confirm_password	Yes (client only)	Prevents typos
email	Optional	Contact info; uniqueness checked if provided

Change Password (components/profile/profile-page.tsx)

Field	Required?
oldPassword	Yes
newPassword	Yes (policy)
confirmPassword	Yes (client match check)

Backend also enforces password history (cannot reuse last 5).

Users Panel (components/settings/users-panel.tsx)

Field	Create	Edit
username	Required	—
password	Required (policy)	Optional (policy if provided)
full_name	Required	Required
email	Optional	Optional
is_active, role_id	Optional	Optional

2. Settings / Admin

Roles & Permissions

Form	Required	Optional
------	----------	----------

Role create/edit	name	description , permission_ids
Permission create/edit	name	description

Statuses (components/settings/statuses-panel.tsx)

Field	Required?	Why
status_name	Yes	Display label
status_type (category)	Yes	Groups statuses (orders, projects, etc.)
color	Yes (must be valid hex)	UI badge rendering
description	Optional	

Entity List / Hierarchy Definitions (components/settings/hierarchy-panel.tsx)

Field	Required?	Why
name	Yes	Registered entity name used across inventory/hardware
hierarchy_type	Yes	system/subsystem/module/unit/component
parent_id	Required when level $\neq$ system	Tree structure
abbreviation	Optional	Used in PN/SN templates

Hierarchy Configuration (components/settings/hierarchy-config-panel.tsx)

Field	Required?	Why
name	Yes, unique	Config identity
code	Yes (backend)	Unique config code
product_types	Yes, ≥ 1	Product variants per config

Tree nodes	≥1 node, each with assigned entity	Defines hardware tree template
description	Optional	

Backend also validates: unique product codes, valid parent/child relationships, build_from_children nodes must have children, entity names must exist in Entity List.

Security Settings

No field-level validation — numeric toggles saved directly; backend applies password policy rules.

3. CRM — Customers & Orders

Customers (app/ (dashboard) /customers/page .tsx)

Field	Create	Edit	Backend
name	Required	Required	Required
status_id	Required (frontend)	Optional (frontend)	Optional
customer_code , organization_type , contact fields, email , phone , country , etc.	Optional	Optional	Optional

Mismatch: Frontend requires status_id on create; backend allows null .

Recommendation: Keep status_id required on create for workflow consistency, but align edit validation (currently only name required on edit).

Orders (app/ (dashboard) /orders/page .tsx , customers/[id]/page .tsx)

Field	Frontend	Backend	Should be
title	Required	Required	Required
order_date	Required	Required	Required

currency	Required	Default "PKR"	Required (or default client-side)
status_id	Required	Optional	Required on create (workflow)
customer_id	Not validated on orders page	Required	Required — gap on /orders page
order_number	Optional (UI field)	Auto-generated	Optional (backend generates)
description, contract_number, po_number, delivery_date, total_value, project_manager, remarks	Optional	Optional	Optional

Why customer_id matters: Every order belongs to a customer. On the customer detail page, customer_id is pre-set when opening the create dialog; on the standalone orders page it is selectable but **not validated**, so submission can fail with a backend 422.

4. Projects

Create Draft (app/ (dashboard) /projects/page.tsx)

Field	Required?	Why
name	Yes	Project identity
hierarchy_config_id	Yes	Defines hardware tree template
order_id	Yes	Links project to commercial order

<code>flight_count</code>	Yes , ≥ 1	Scope definition
<code>sdls_counts_by_flight[]</code>	Yes , length = <code>flight_count</code> , each ≥ 1	Per-flight SDLS counts
<code>product_type</code>	Yes (backend)	Not always validated on frontend — sent from form default
<code>description</code> , <code>start_date</code> , <code>end_date</code> , <code>owner_id</code>	Optional on create	Can be set later
<code>is_existing_project</code>	Optional flag	Role-gated (Admin/PD/HM only)

Edit Project

Field	Frontend	Backend (<code>ProjectUpdate</code>)
<code>name</code> , <code>owner_id</code> , <code>status_id</code> , <code>order_id</code> , <code>start_date</code> , <code>end_date</code>	All required	All optional

Mismatch: Frontend is stricter than backend on edit. **Recommendation:** Keep frontend strictness — these fields drive project lifecycle and reporting.

Config Change Wizard (`components/projects/config-change-wizard.tsx`)

Field	Required?
<code>target_hierarchy_config_id</code>	Yes
<code>product_type</code>	Yes
<code>reason</code>	Yes
<code>flight_count</code> , <code>sdls_per_flight</code>	Optional (coerced with <code>Number()</code>)

5. Hardware Hierarchy (System → Component)

List-page create dialogs

Entity	Required fields	Why
System	name , project_id , status_id	Anchors hardware to project + workflow state
Subsystem	name , system_id	Parent linkage
Module	name , subsystem_id	Parent linkage
Unit	name , module_id	Parent linkage (frontend); backend allows optional module_id
Component	name , unit_id	Parent linkage (frontend); backend allows optional unit_id

Backend also requires: name must exist in Entity List registry.

Mismatch: Backend allows orphan units/components (module_id / unit_id optional).

Recommendation: Keep frontend parent requirements — orphan nodes break the hierarchy tree.

EntityForm (dashboard create/edit) — components/entity-form.tsx

Field	Create	Edit
name	Required	Required
Parent (project_id , system_id , etc.)	Required	Hidden/disabled
id (status)	Required	Required
description , partnumber	Optional	Optional
inventory_instance_id	Optional	—
Install fields (installation_date , installed_by_id , picture_url)	Optional	Optional (systems only)

Runtime rule: If serialized inventory is selected, serial number is required via `createHierarchyEntityFromForm()` even though the field isn't marked `required`.

Assign Developer (`components/hierarchy/assign-developer-dialog.tsx`)

Field	Required?
<code>developerId</code>	Yes (positive integer; blocked if already issued)

6. Inventory

Central logic in `hooks/use-inventory-entity-form.ts`:

```
const validateCreate = useCallback(): string | null => {
  const usesInstances = inventoryUsesInstances(selectedEntityType);
  const isHierarchy = context === 'hierarchy';
  if (!formData.name.trim() || (!isHierarchy && !usesInstances &&
!formData.location.trim())) {
    return `Please fill in required fields:
${entityLabel(selectedEntityType)} category${
      usesInstances || isHierarchy ? '' : ' and Location'
    }`;
  }
  if (usesInstances && !formData.part_number.trim()) {
    return 'Part number is required for serialized inventory';
  }
  // ... location for serialized non-component, quantity for
  components
}
```

Field	When required	Why
<code>name</code> (entity category)	Always	Must match Entity List

inventory_type	Always (UI)	Determines serialized vs bulk behavior
part_number	Serialized types	Traceability / labeling
location	Bulk inventory; serialized non-component (inventory context)	Physical tracking
quantity	Components with quantity support, > 0	Stock count
serial_number	At issue time for serialized items	Unit-level tracking
description, oem_name, sku, holder_user_id, dates, install fields	Optional	Metadata

Backend: name + inventory_type required; part number required for non-component; quantity ≥ 1 ; location defaults to "Warehouse" if empty.

Issue to Developer (components/inventory-issue-dialog.tsx)

Field	Required?	Why
Developer	Yes	Accountability
Serial (instance_id)	Yes if serialized	Which physical unit
Quantity	≥ 1 , $\leq$ available (bulk)	Stock control
Signature (digital or hard-copy)	Yes	Issuance audit trail
Notes	Optional	

Issuance Remarks / Return Decision

Field	Required?
notes / admin remarks	Yes (trimmed)

Shortage Receive

Field	Required?
quantity	≥1
part_number	Yes
serial_number, location	Optional

7. Maintenance

Maintenance Case

(components/maintenance/MaintenanceCaseDialog.tsx)

Field	Create	Edit
project_id	Required	Hidden
description	Required	Hidden
status	Shown	Required
resolution_notes	—	Optional

Backend: description + project_id required on create.

Maintenance Logs

(app/(dashboard)/maintenanceLogs/page.tsx)

Field	Required?	Why
serial_number	Yes	Lookup key
performed_by	Yes	Accountability

notes	Yes	Audit record
maintenance_type	Default preventive	

Backend only requires entity_id (set via lookup).

Resolve Fault (components/maintenance/resolve-fault-dialog.tsx)

Field	Required?
resolutionType	Yes
Replacement serial	Yes if type = REPLACED and stock exists
notes	Optional

8. Attachments & Media

Attachment Upload (components/attachment-upload-dialog.tsx)

Field	Required?
file	Yes when requireFile=true
attachment_type	Default "other"
description	Optional

Backend: owner_type , owner_id , file required; owner_type must be valid enum.

Frontend vs Backend — Key Gaps

Form	Issue	Recommendation
Orders (/orders)	customer_id not validated client-side	Require customer_id on create/update

Customers	<code>status_id</code> required on create but optional on backend	Keep required on create; consider backend default status
Customers edit	Only <code>name</code> validated	Consider requiring <code>status_id</code> if workflow depends on it
Systems	Frontend requires <code>status_id</code> ; backend optional	Keep frontend requirement
Project edit	Frontend requires 6 fields; backend all optional	Frontend strictness is correct for UX
Units/Components	Frontend requires parent; backend allows orphan	Keep frontend requirement
EntityForm	<code>!formData[field]</code> treats <code>0</code> as invalid	Use explicit empty checks
No schema layer	Rules duplicated/inconsistent across ~40 forms	Consider adopting the installed Zod + react-hook-form

Recommended Required vs Optional Summary

Always required (identity / workflow anchors)

- User: `username`, `password` (create), `full_name`
- Customer: `name`
- Order: `title`, `order_date`, `customer_id`, `status_id` (create)
- Project create: `name`, `order_id`, `hierarchy_config_id`, `flight_count`, SDLS counts
- Hardware: `name` + parent FK + Entity List registration
- Inventory: `name`, `inventory_type`; conditional `part_number`, `location`, `quantity`
- Maintenance case: `project_id`, `description`
- Issuance: developer, signature, serial (if serialized)

Should stay optional (metadata / enrichment)

- Contact info: email, phone, address, website
- Descriptions, remarks, notes (except where explicitly required for audit)

- Install metadata: `installation_date`, `installed_by_id`, `picture_url`
- Order: `contract_number`, `po_number`, `delivery_date`, `total_value`, `project_manager`
- Project create: `description`, dates, `owner_id` (can default)
- Inventory: OEM, SKU, shelf life, holder

Conditionally required (business rules)

- Password policy fields (dynamic from settings)
- Serial number (serialized inventory at issue/create-from-stock)
- Replacement serial (fault resolution type = REPLACED)
- Entity List name match (all hardware/inventory names)
- Hierarchy config tree integrity (parent/child, product types)
- Signature payload (digital issuance)

Why Validation Exists Where It Does

Rule	Business reason
Entity List name check	Ensures standardized hardware taxonomy across projects
Parent FK on hierarchy	Maintains system → subsystem → module → unit → component tree
Password policy	Security compliance
Issuance signature	Audit trail for inventory custody
SDLS/flight counts	Defines project scope for hierarchy generation
<code>customer_id</code> on orders	Commercial relationship tracking
Part number on serialized inventory	Labeling, traceability, shortage matching
Status on create	Workflow state machine (Created → Active → etc.)

If you want to go further, I can produce a **field-by-field matrix** for a specific module (e.g. inventory only), or a **prioritized fix list** for the frontend/backend mismatches. Switch to Agent mode if you'd like those implemented.